Cosmos SDK EVM 구현체 비교: 아키텍처 접근법의 진화

1. 들어가며

Cosmos SDK 위에서 Ethereum Virtual Machine(EVM)을 구동하려는 시도는 2016년 **Ethermint** 프로젝트에서 시작되었습니다. 그러나 "EVM + CometBFT"라는 단순해 보이는 조합은 예상보다 훨씬 복잡한 기술적 도전을 수반했습니다.

> "The high-performance architecture of Octane has only been made possible in the last six months thanks to recent advancements in ABCI++ and the Ethereum Engine API. It has unlocked a completely new class of performance that no previous generation EVM implementation was able to achieve."
> — Tyler Tarsi, CTO of Omni Labs

2024년을 기점으로, Cosmos 생태계에서는 EVM 통합에 대한 **두 가지 상반된 접근법**이 등장했습니다. 하나는 EVM을 Cosmos SDK 모듈로 임베드하는 **전통적 방식**이고, 다른 하나는 Ethereum의 Engine API를 활용해 실행 계층을 완전히 분리하는 **새로운 방식**입니다.

이 글에서는 Cosmos EVM(Ethermint), Berachain(Polaris → BeaconKit), Omni(Octane), Initia(MiniEVM) 등 주요 구현체들의 아키텍처 설계 결정을 비교 분석하고, 각 접근법의 장단점과 보안 고려사항을 살펴봅니다.

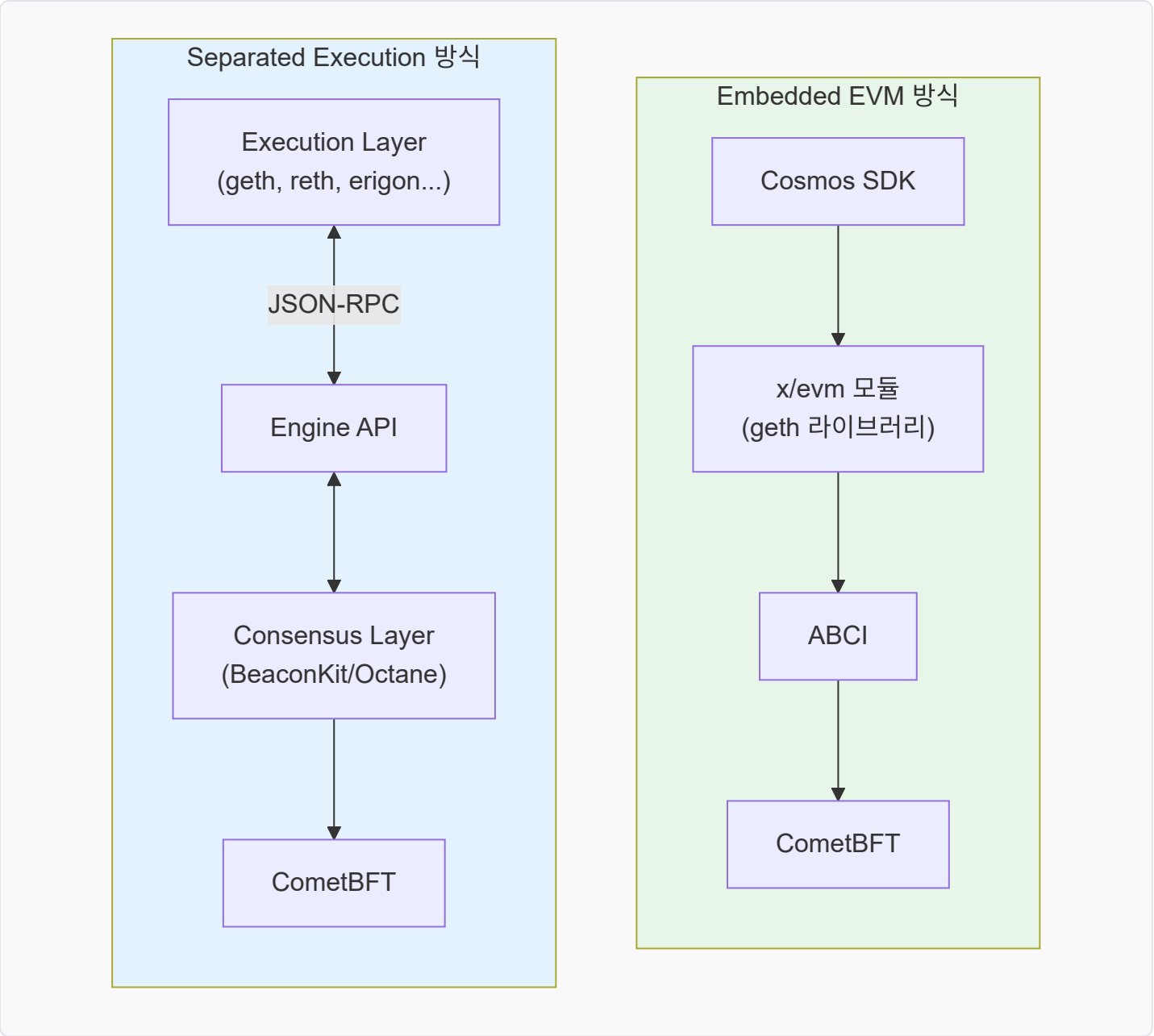
---

2. EVM 구현체 분류

2.1 분류 기준

Cosmos SDK 기반 EVM 구현체들은 **실행 계층의 통합 방식**에 따라 크게 두 가지로 분류됩니다:

분류	접근 방식	대표 프로젝트
Embedded EVM	EVM을 Cosmos SDK x/evm 모듈로 구현	Cosmos EVM, Polaris, MiniEVM
Separated Execution	Engine API로 EVM 클라이언트 분리	Octane, BeaconKit



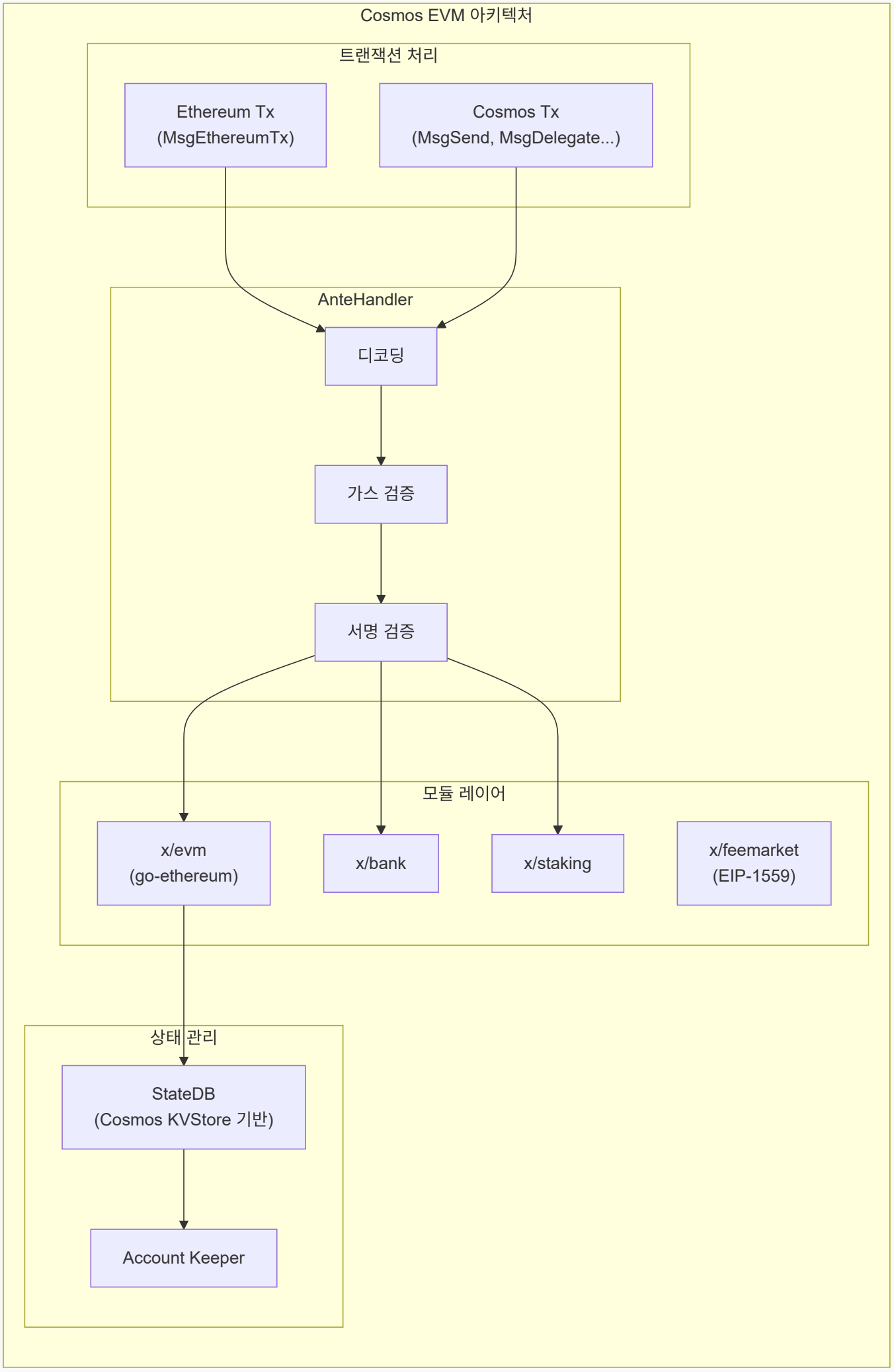
---

3. Embedded EVM 방식

3.1 Ethermint / Cosmos EVM

Ethermint은 EVM을 Cosmos SDK의 x/evm 모듈로 구현한 최초의 프로젝트입니다. 2022년 Evmos로 메인넷을 런칭했고, 2025년 Cosmos EVM으로 오픈소스화되었습니다.

핵심 아키텍처


Cosmos EVM 아키텍처
트랜잭션 처리
Ethereum Tx
(MsgEthereumTx)
Cosmos Tx
(MsgSend, MsgDelegate...)
AnteHandler
디코딩
가스 검증
서명 검증
모듈 레이어
x/evm
(go-ethereum)
x/bank
x/staking
x/feemarket
(EIP-1559)
상태 관리
StateDB
(Cosmos KVStore 기반)
Account Keeper

> "Cosmos EVM enables the full composability and modularity of the Cosmos SDK. It includes standard modules from the Cosmos SDK that work side to side with EVM-specific modules."
> — Cosmos EVM Documentation

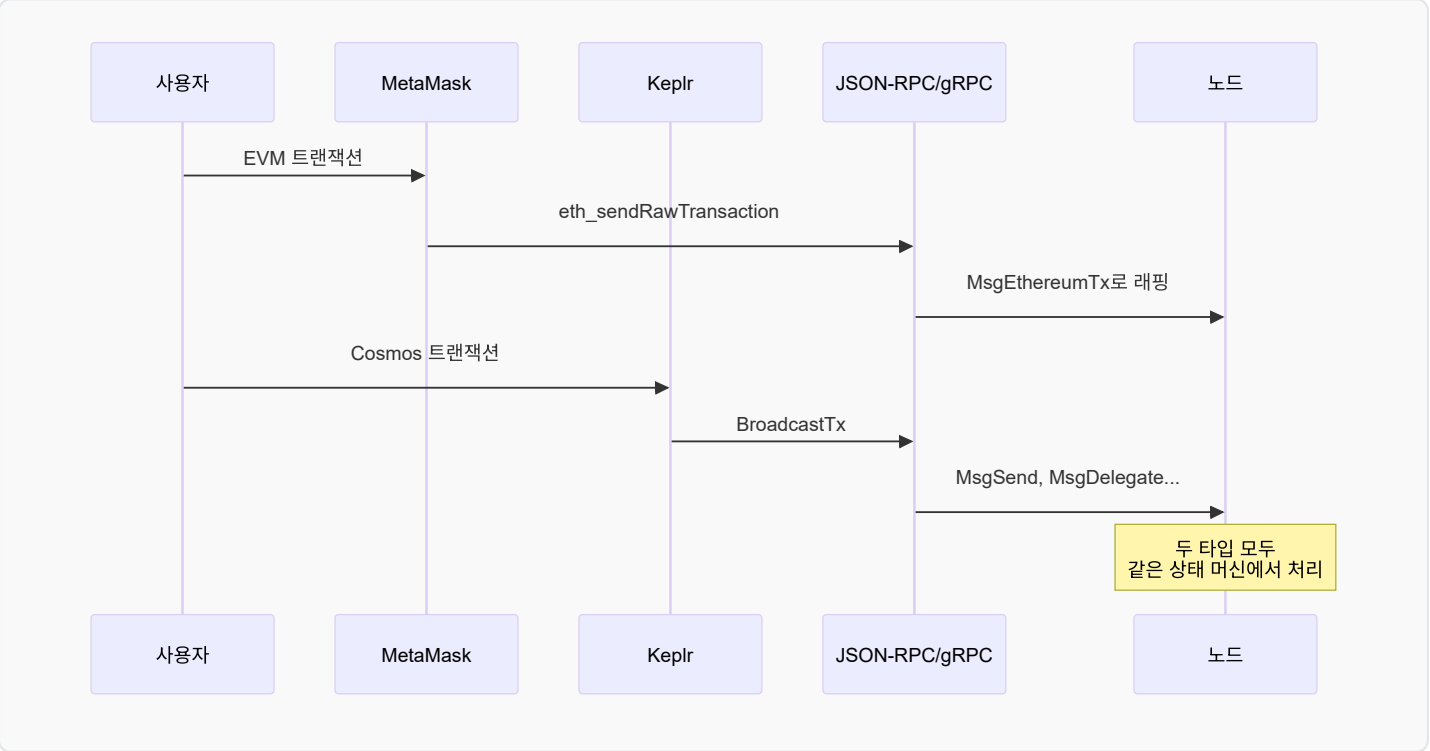
핵심 설계 결정

1. geth를 라이브러리로 사용

```
// go-ethereum의 EVM을 직접 임포트
import (
    "github.com/ethereum/go-ethereum/core/vm"
    "github.com/ethereum/go-ethereum/core/state"
)
```

Cosmos EVM은 go-ethereum(geth)의 EVM 구현을 **라이브러리**로 임포트합니다. 별도의 프로세스가 아닌, 동일한 바이너리 내에서 EVM 코드가 실행됩니다.

2. 이중 트랜잭션 타입



Cosmos EVM은 **두 가지 트랜잭션 형식**을 모두 지원합니다:

- `MsgEthereumTx`: Ethereum RLP 인코딩된 트랜잭션
- Cosmos SDK 메시지: `MsgSend`, `MsgDelegate`, `MsgVote` 등

3. StateDB 커스텀 구현

가장 복잡한 부분입니다. Ethereum의 StateDB 인터페이스를 Cosmos SDK의 KVStore 기반으로 재구현해야 합니다.

```
// Ethermint StateDB 구현 (간략화)
type StateDB struct {
    keeper        *Keeper
    ctx           sdk.Context
    stateObjects  map[common.Address]*stateObject
    journal       *journal  // 스냅샷/리버트 지원
}

func (s *StateDB) SetState(addr common.Address, key, value common.Hash) {
    stateObject := s.getOrNewStateObject(addr)
    stateObject.SetState(key, value)
    s.journal.append(storageChange{...})  // 저널에 기록
}
```

직면한 문제점

문제 1: 깊은 Context Stack의 성능 저하

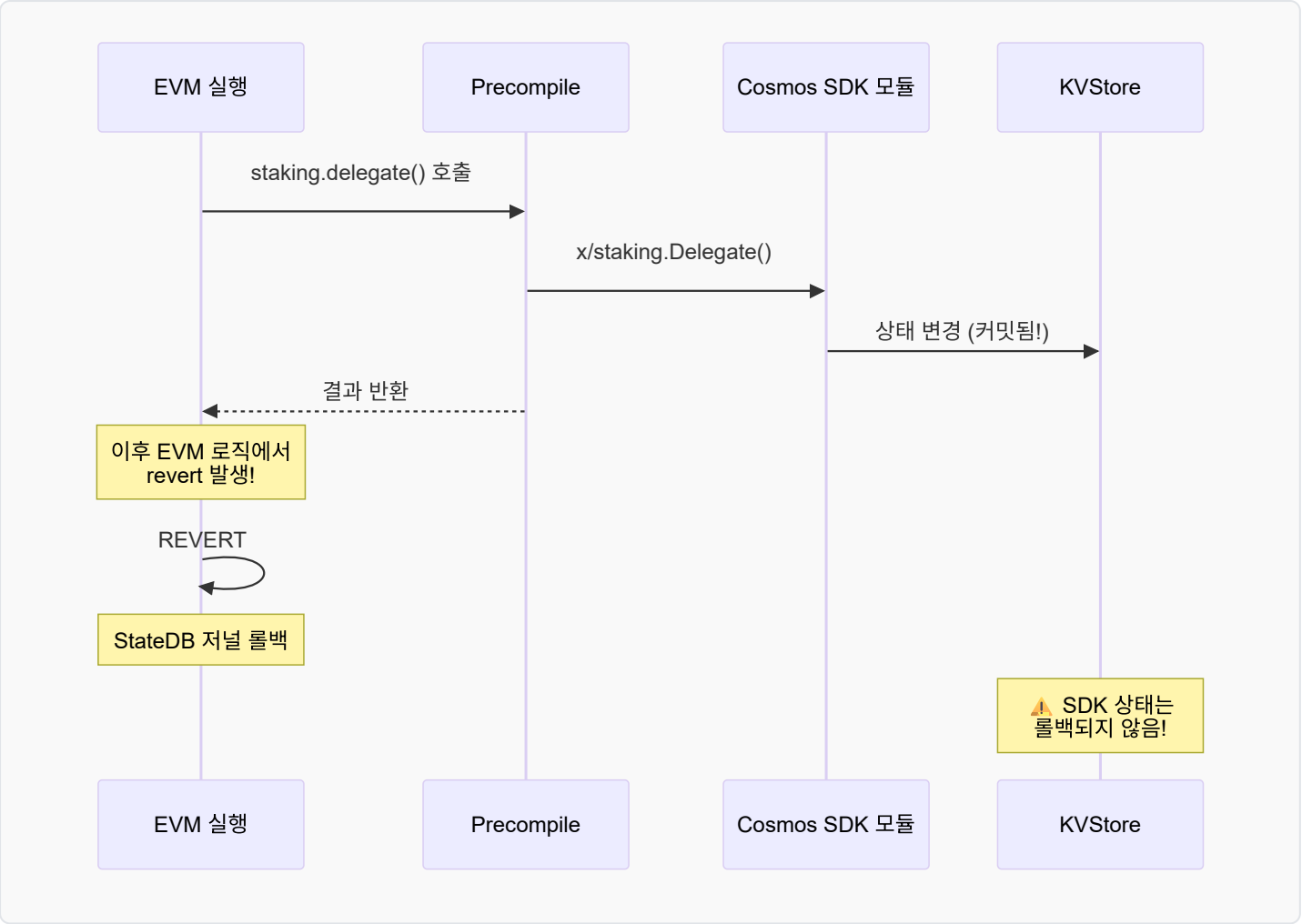
Cosmos SDK의 CacheKVStore는 중첩된 컨텍스트를 지원하지만, 깊은 스택에서 **O(N×M) 복잡도** 문제가 발생합니다.

> "The bottleneck is the slowness of deep context stack, there's no easy way around it other than refactor the StateDB to use journal logs like what go-ethereum does."
> — Ethermint PR #729

2021년 벤치마크에서 단순한 메시지 콜 체인이 **30초**가 걸리는 문제가 발견되어, go-ethereum 스타일의 저널 기반 StateDB로 전면 리팩토링되었습니다.

문제 2: Stateful Precompile의 원자성

EVM에서 Cosmos SDK 모듈을 호출하는 **Stateful Precompile**에서 심각한 문제가 발생합니다:



> "If we were to call a module's logic through a stateful precompile and NOT use the journal semantics, even though the EVM transition reverted, the KVStore changes would persist."
> — Ethermint GitHub Issue #1363

이 문제로 인해 Berachain 팀은 **Cosmos SDK 모듈 로직을 StateDB 저널 시맨틱에 맞게 완전히 재작성**해야 했습니다.

문제 3: 가스 마켓 불일치

Cosmos와 EVM은 서로 다른 가스 모델을 사용합니다. 이로 인해 2022년 이후 여러 심각한 보안 취약점이 발견되었습니다:

취약점	영향	원인
Cronos 수수료 도난	트랜잭션 수수료 탈취	AnteHandler 가스 처리 불일치
Ethermint AnteHandler 우회	가스 없이 트랜잭션 실행	EVM/Cosmos 가스 분리 미흡

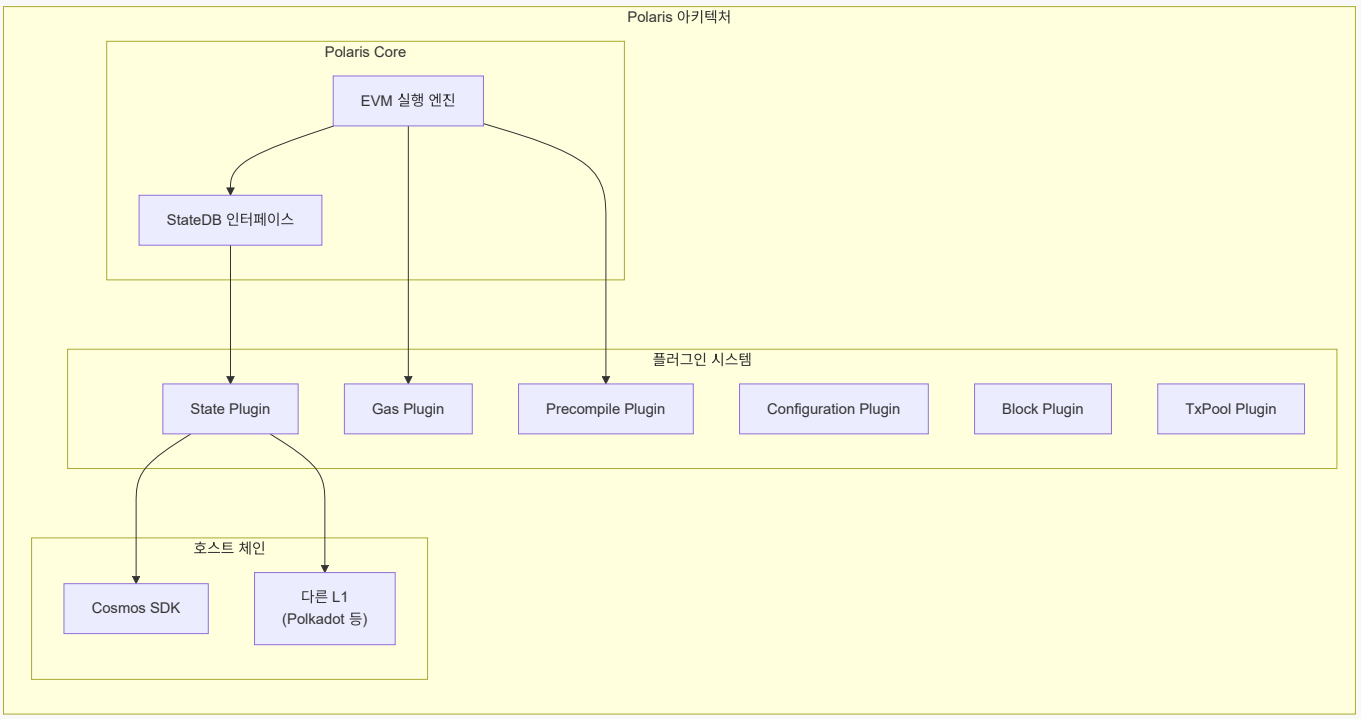
> "Multiple critical vulnerabilities have been disclosed in EVMOS-style chains, all involving transaction fee manipulation."
> — CertiK Research

3.2 Berachain Polaris (V1)

Berachain V1은 **Polaris**라는 자체 EVM 프레임워크를 개발했습니다.

> "Polaris is a general purpose blockchain framework that allows for the integration of the Ethereum Virtual Machine (EVM) into any underlying networking or consensus layer."
> — Polaris Documentation

Polaris의 차별점



Polaris의 핵심 아이디어는 **플러그인 아키텍처**입니다. 호스트 체인은 6개의 플러그인 인터페이스만 구현하면 EVM을 통합할 수 있습니다.

```
// Polaris 플러그인 인터페이스 (간략화)
type Plugin interface {
    Configuration() ConfigPlugin
    State() StatePlugin
    Block() BlockPlugin
    Gas() GasPlugin
    Precompile() PrecompilePlugin
    TxPool() TxPoolPlugin
}
```

Polaris의 한계

Berachain 팀은 Artio 테스트넷에서 심각한 성능 문제를 경험했습니다:

> "One major challenge with Polaris was that the Cosmos SDK couldn't handle the transaction volume Berachain received. Additionally, the forked EVM client had compatibility issues."
> — Berachain Blog

주요 문제점:

1. **CometBFT 멤풀 병목**: 높은 트래픽에서 멤풀이 혼잡 유발
2. **Cosmos SDK 오버헤드**: 모듈 간 의존성이 성능 저하 초래
3. **EVM 호환성**: 포크된 EVM 클라이언트 유지 보수 부담

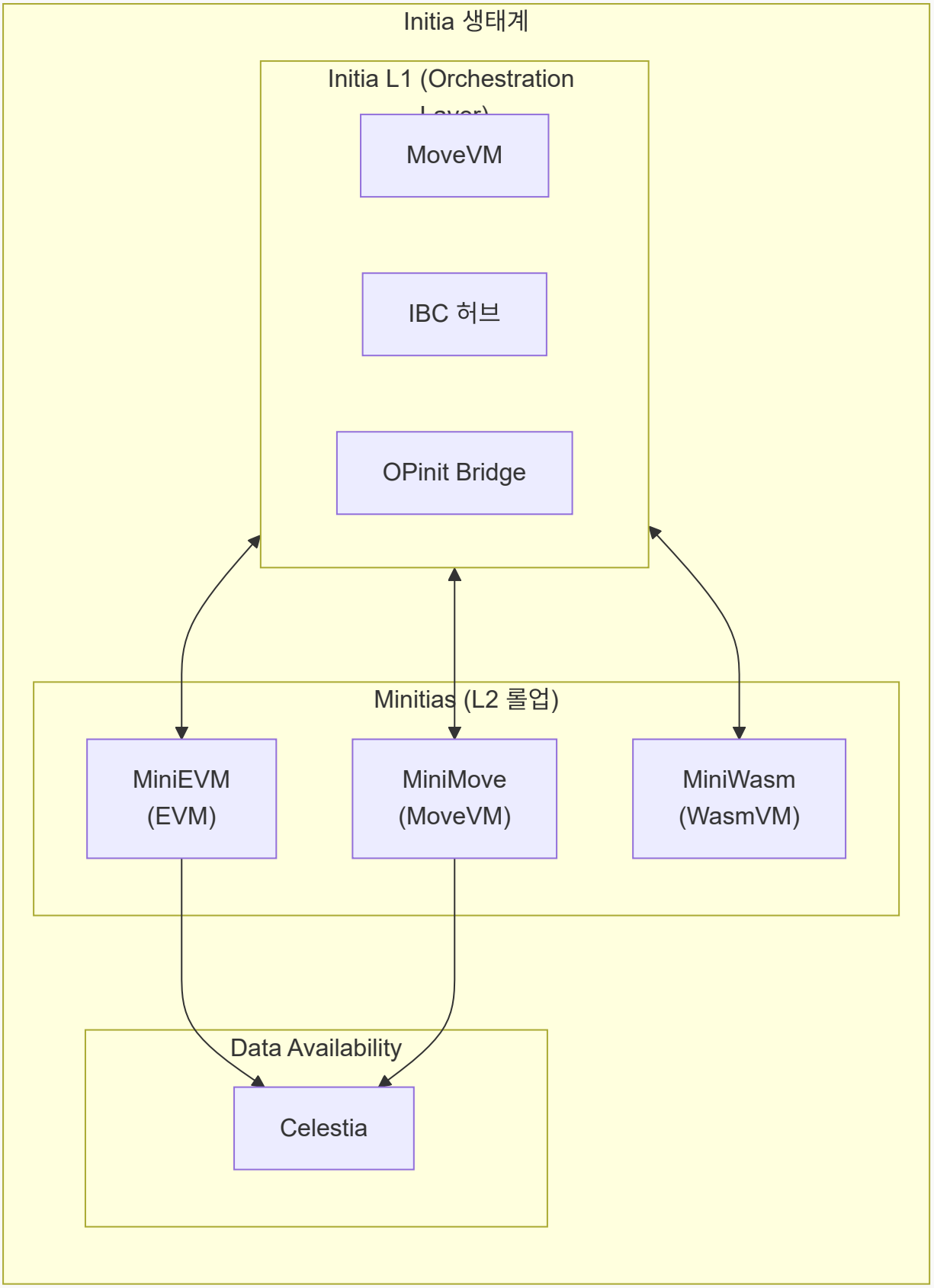
이 경험은 Berachain을 완전히 새로운 접근법인 **BeaconKit**으로 이끌었습니다.

3.3 Initia MiniEVM

Initia는 **OPinit Stack**이라는 Cosmos SDK 기반 Optimistic Rollup 프레임워크를 개발했고, MiniEVM은 이 위에서 동작하는 EVM 롤업입니다.

> "MiniEVM is an optimistic rollup consumer chain powered by EVM, designed to simplify the process of bootstrapping an L2 network."
> — Initia GitHub

MiniEVM의 특징



핵심 특징:

- **Optimistic Rollup**: L1(Initia)에 상태 루트만 제출, fraud proof로 보안
- **VM 무관**: 동일한 OPinit Stack 위에 EVM, MoveVM, WasmVM 모두 지원
- **500ms 블록 타임**: L1 컨센서스 없이 빠른 트랜잭션 확정

Cosmos Precompile: Dispatch 패턴

MiniEVM의 독특한 특징은 **Cosmos Precompile**입니다. EVM 스마트 컨트랙트에서 Cosmos SDK 메시지를 직접 실행할 수 있습니다.

```
// MiniEVM Cosmos Precompile 사용 예
import { ICosmos } from "@initia/evm-precompiles/i_cosmos/ICosmos.sol";

contract CosmosInterop {
    function sendTokens(string memory toAddress, uint256 amount) external {
        string memory msg = string(abi.encodePacked(
            '{"@type":"/cosmos.bank.v1beta1.MsgSend",',
            '"from_address":"', msg.sender, '",',
            '"to_address":"', toAddress, '",',
            '"amount":[{"denom":"stake","amount":"', amount, '"}]}'
        ));

        // EVM 실행 완료 후 Cosmos 메시지 실행
        ICosmos.COSMOS_CONTRACT.execute_cosmos(msg);
    }
}
```

그러나 이 패턴은 여러 보안 취약점을 노출했습니다:

취약점	설명
가스 리밋 우회	dispatch된 EVM 호출이 명시적 가스 리밋 무시
스택 오버플로우 DoS	가스 미부과 상태에서 무한 호출 가능
콜백 에러 미전파	중첩 호출의 에러가 부모에게 전달 안 됨

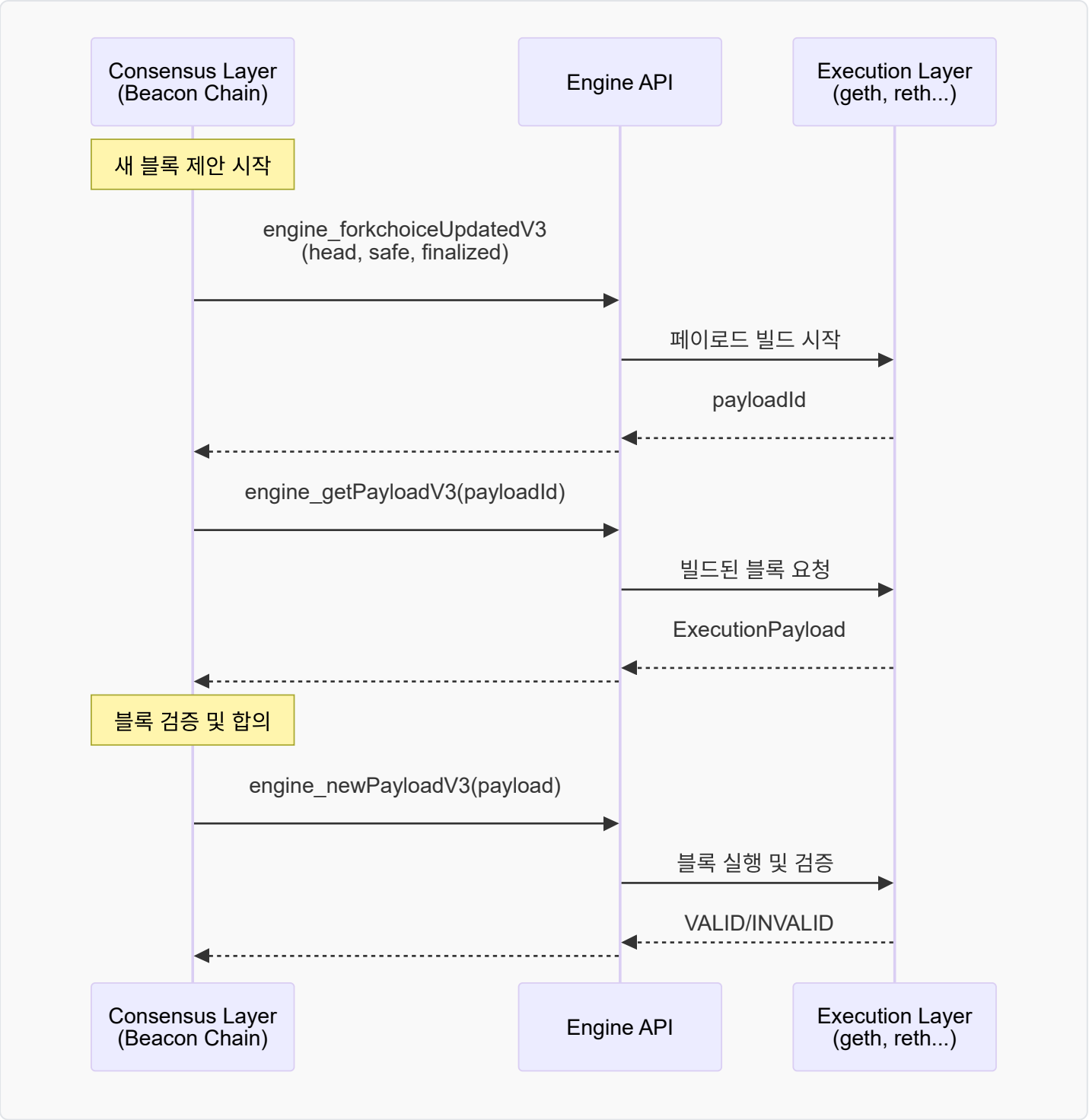
> "Explicit gas limit on low-level Solidity calls can be bypassed by dispatched EVM calls via the custom Cosmos precompile."
> — Code4rena Audit Report

---

4. Separated Execution 방식

4.1 패러다임 전환: Engine API

2022년 Ethereum Merge 이후, **Engine API**라는 새로운 인터페이스가 등장했습니다. 이는 컨센서스 레이어(Beacon Chain)와 실행 레이어(geth)를 JSON-RPC로 연결합니다.



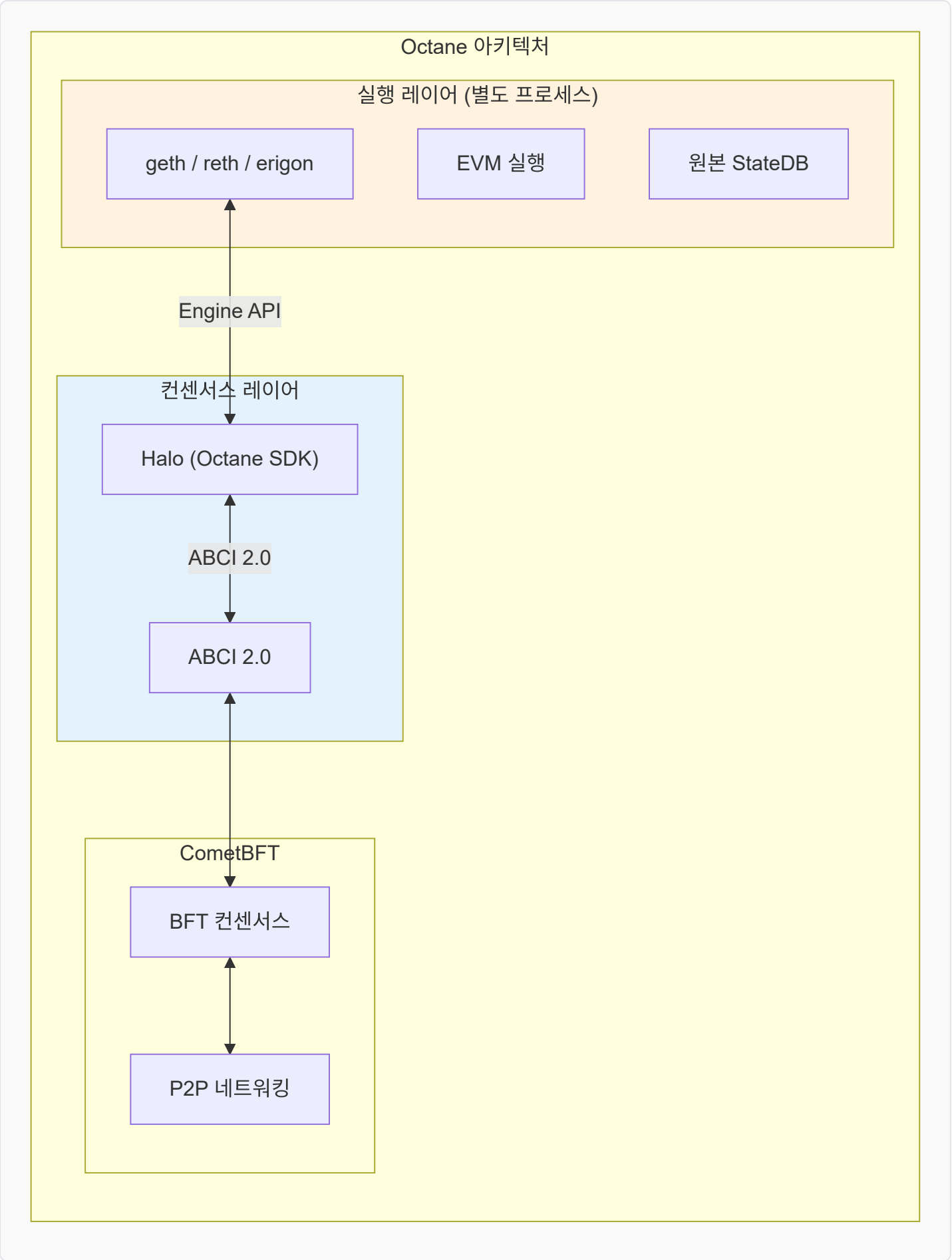
Cosmos 생태계의 일부 팀들은 이 아키텍처를 CometBFT에 적용하기 시작했습니다.

4.2 Omni Octane

Octane은 Omni Network가 개발한 EVM 프레임워크로, **Engine API + ABCI 2.0**을 결합합니다.

> "Octane was designed to address the scalability and maintenance issues encountered with Ethermint, the previous standard for running the EVM on CometBFT."
> — Omni Documentation

Octane 아키텍처



핵심 혁신: Engine API 최초의 Cosmos 구현

> "Octane is the first consensus implementation of the Engine API besides Ethereum itself (the Beacon Chain)."

> — Omni Blog

Engine API 사용의 이점:

1. **EVM 클라이언트 다양성**: 수정 없이 geth, reth, erigon 등 사용 가능
2. **유지보수 부담 감소**: EVM 업그레이드를 자동으로 반영
3. **완벽한 EVM 호환성**: 포크된 EVM이 아닌 원본 사용

```
// Octane의 Engine API 호출 (개념적)
func (o *Octane) PrepareProposal(req *abci.RequestPrepareProposal) {
    // 1. Engine API로 페이로드 빌드 시작
    payloadID := o.engineClient.ForkchoiceUpdated(head, safe, finalized)

    // 2. 빌드된 페이로드 가져오기
    payload := o.engineClient.GetPayload(payloadID)

    // 3. CometBFT 블록에 포함
    return &abci.ResponsePrepareProposal{
        Txs: [][]byte{payload.Marshal()},
    }
}
```

ABCI 2.0 (ABCI++) 활용

Octane은 ABCI 2.0의 새로운 메서드들을 적극 활용합니다:

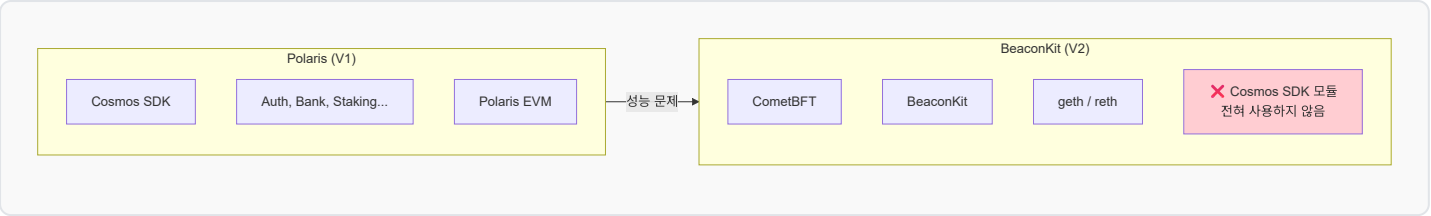
ABCI 2.0 메서드	용도
`PrepareProposal`	Engine API로 블록 빌드 시작
`ProcessProposal`	제안된 블록 검증
`ExtendVote`	크로스체인 메시지 attestation
`VerifyVoteExtension`	attestation 검증

4.3 Berachain BeaconKit (V2)

Polaris의 한계를 경험한 Berachain 팀은 **BeaconKit**이라는 완전히 새로운 프레임워크를 개발했습니다.

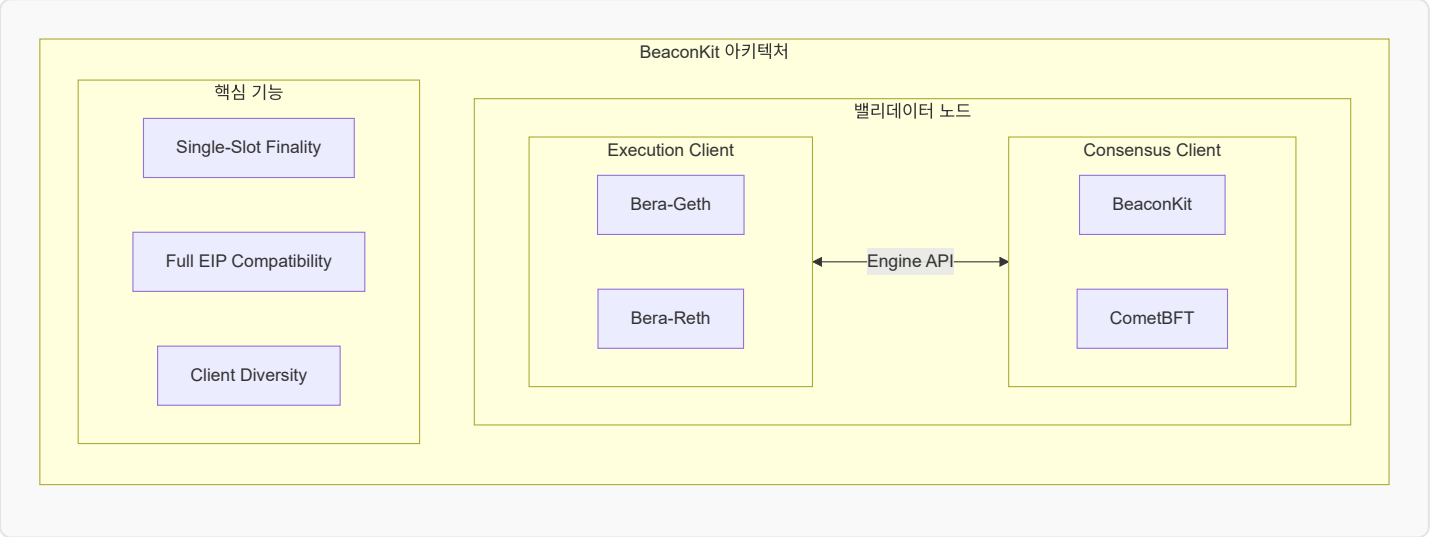
> "BeaconKit is a modular framework for building EVM consensus clients, enabling developers to launch both layer 1 and 2 EVM blockchains with full EIP compatibility, single-slot finality, and much more."
> — Berachain Blog

급진적 결정: Cosmos SDK 모듈 전면 제거



> "BeaconKit uses zero standard Cosmos modules and allows chain developers to inject custom logic on a case-by-case basis. 'Wait, you need AccountKeeper? BankKeeper?? Ughh...' is a common complaint among Cosmos developers."
> — Berachain Blog

BeaconKit 아키텍처



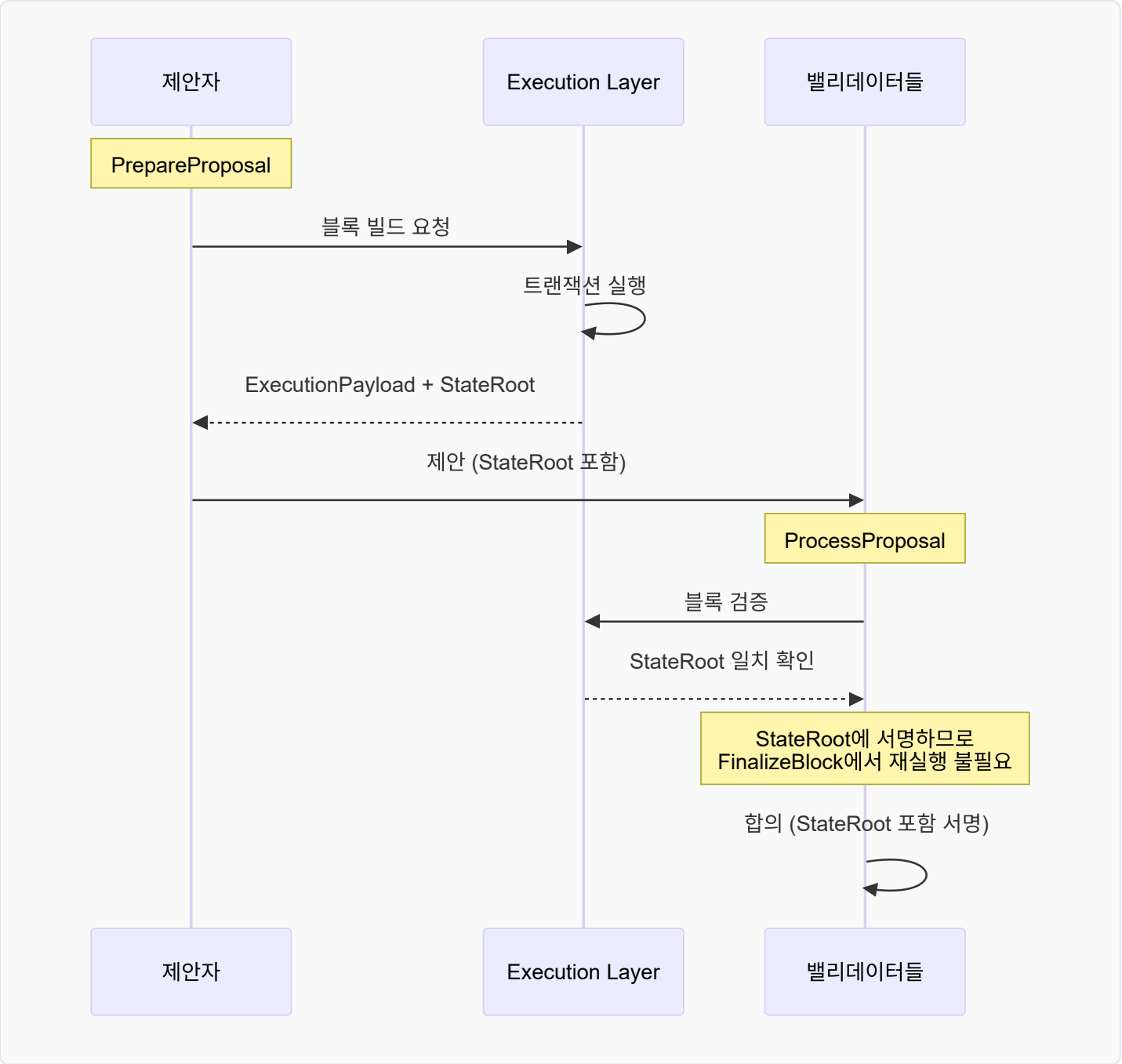
BeaconKit의 핵심 특징:

특징	설명
EVM Identical	수정 없는 geth/reth 사용으로 100% EVM 호환
Single-Slot Finality	CometBFT의 즉시 확정성 (~3초)
Client Diversity	geth, reth, erigon, nethermind, besu 등 지원
Immediate Execution	블록 확정 전 StateRoot 계산 완료 (40% 성능 향상)

Immediate Execution

BeaconKit의 독특한 최적화입니다:

> "By leveraging our custom BeaconBlock on top of the standard CometBFT block, BeaconKit supports immediate execution. This means that BeaconKit chains can enforce that Validators sign over the proposed StateRoot before accepting the block."
> — Berachain Blog



일반적인 Cosmos SDK 체인에서는 `FinalizeBlock`에서 모든 트랜잭션을 재실행해야 하지만, BeaconKit은 이미 검증된 StateRoot를 사용하므로 이 과정을 건너뜁니다.

---

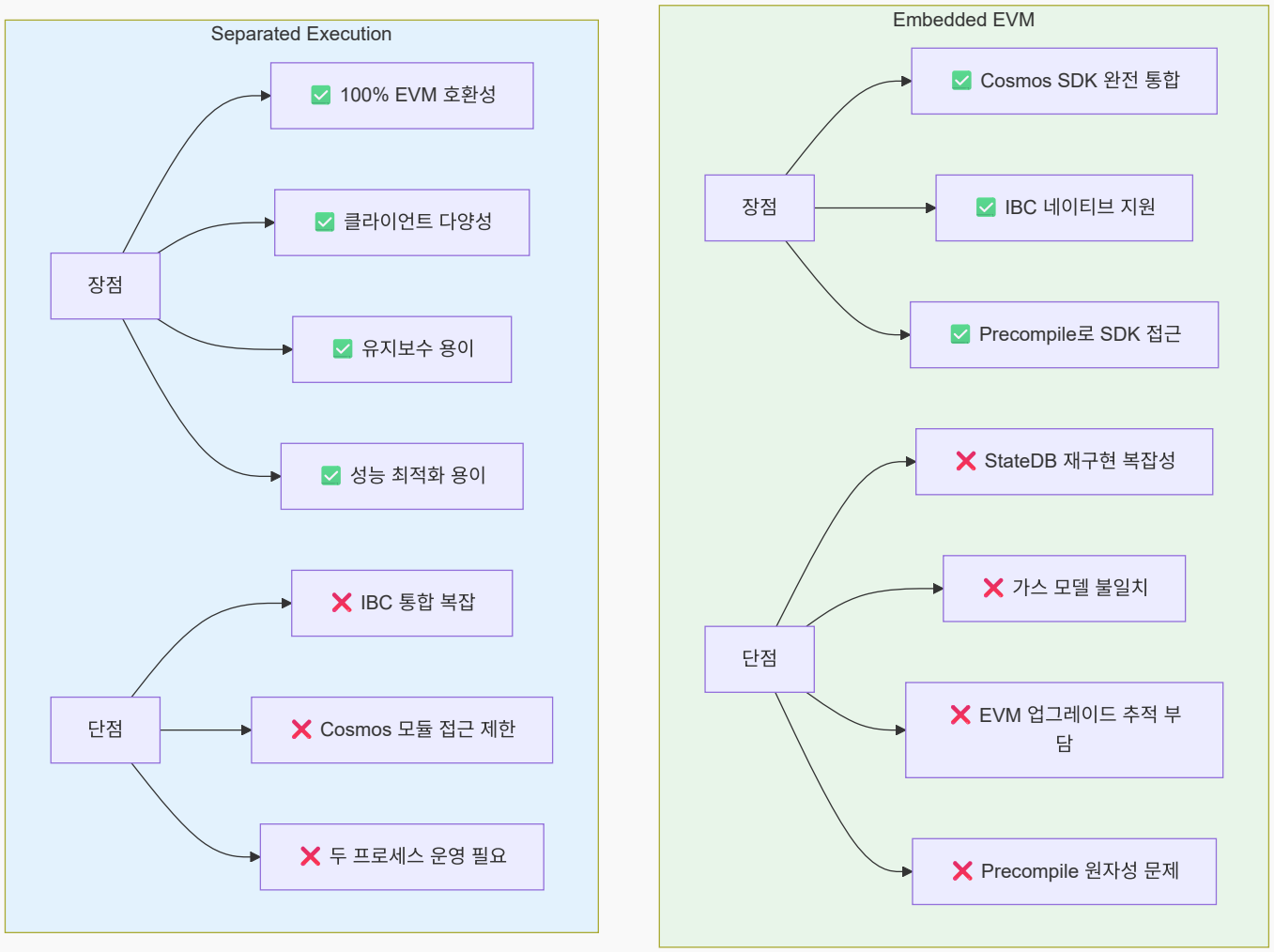
5. 아키텍처 비교 분석

5.1 종합 비교표

항목	Cosmos EVM	Polaris	MiniEVM	Octane	BeaconKit
분류	Embedded	Embedded	Embedded	Separated	Separated
Cosmos SDK 사용	✅ 전체	✅ 전체	✅ 전체	⚠️ 일부	❌ 없음

항목	Cosmos EVM	Polaris	MiniEVM	Octane	BeaconKit
EVM 통합 방식	x/evm 모듈	플러그인	x/evm 모듈	Engine API	Engine API
EVM 클라이언트	geth (라이브러리)	geth (라이브러리)	geth (라이브러리)	geth/reth (프로세스)	geth/reth (프로세스)
IBC 지원	✅ 네이티브	✅ 네이티브	✅ 네이티브	✅ (추가 작업)	⚠️ 제한적
EVM Precompile	✅ 지원	✅ 지원	✅ 지원	❌ 불필요	❌ 불필요
실행 클라이언트 선택	❌	❌	❌	✅	✅
L2 롤업 지원	❌	❌	✅ OPinit	✅ Rollkit	✅ Rollkit

5.2 장단점 분석



5.3 사용 사례별 권장

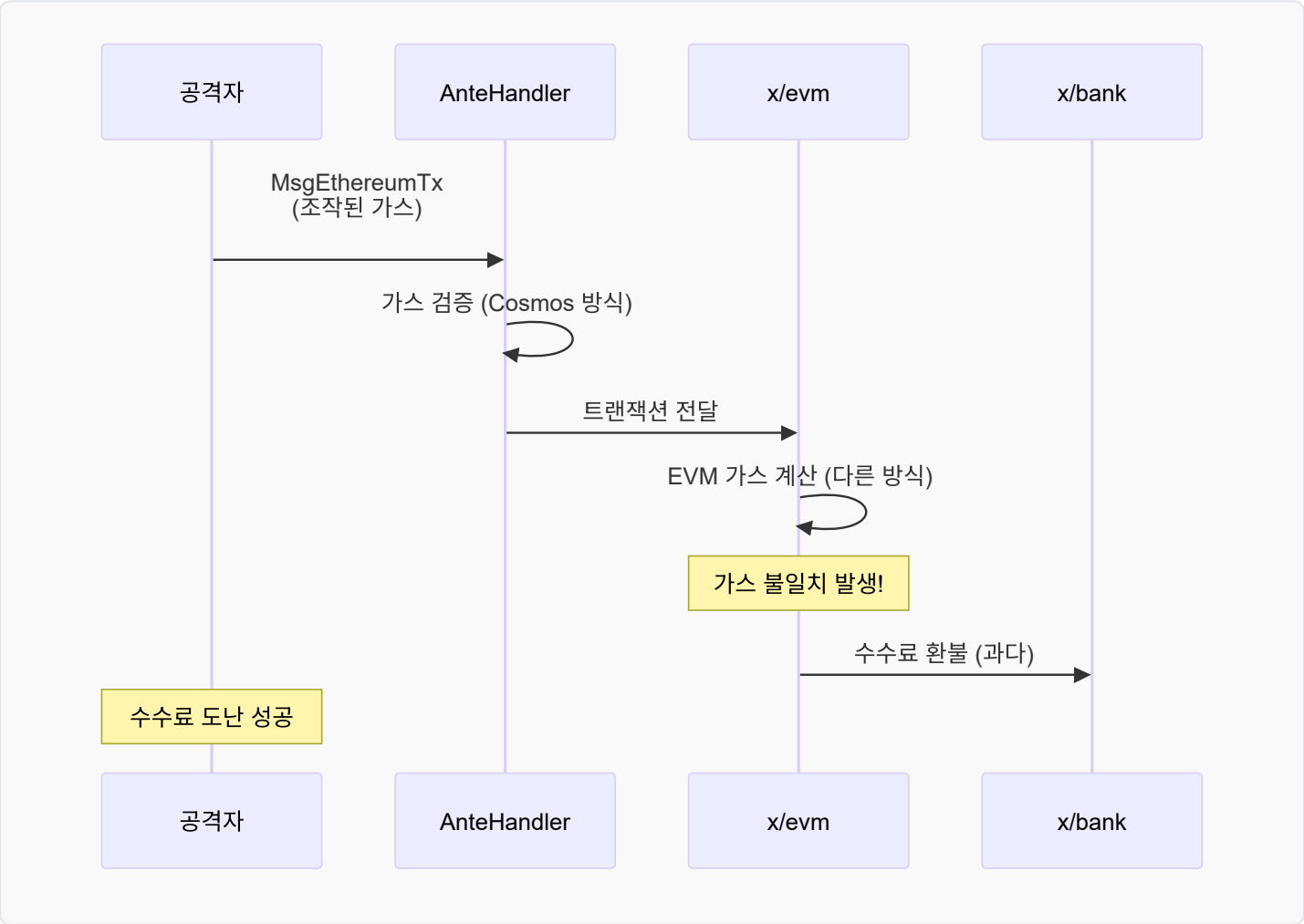
사용 사례	권장 방식	이유
IBC 헤비 사용	Embedded (Cosmos EVM)	네이티브 IBC 지원
DeFi 프로토콜 포팅	Separated (BeaconKit)	100% EVM 호환성
L2 롤업	MiniEVM 또는 BeaconKit	롤업 프레임워크 내장
고성능 L1	Separated (Octane/BeaconKit)	최적화된 실행 분리
Cosmos 모듈 통합	Embedded (Cosmos EVM)	Precompile 활용

---

6. 보안 고려사항

6.1 Embedded EVM의 보안 이슈

AnteHandler 취약점



Precompile 원자성 취약점

```
// 취약한 패턴
function maliciousCall() external {
```

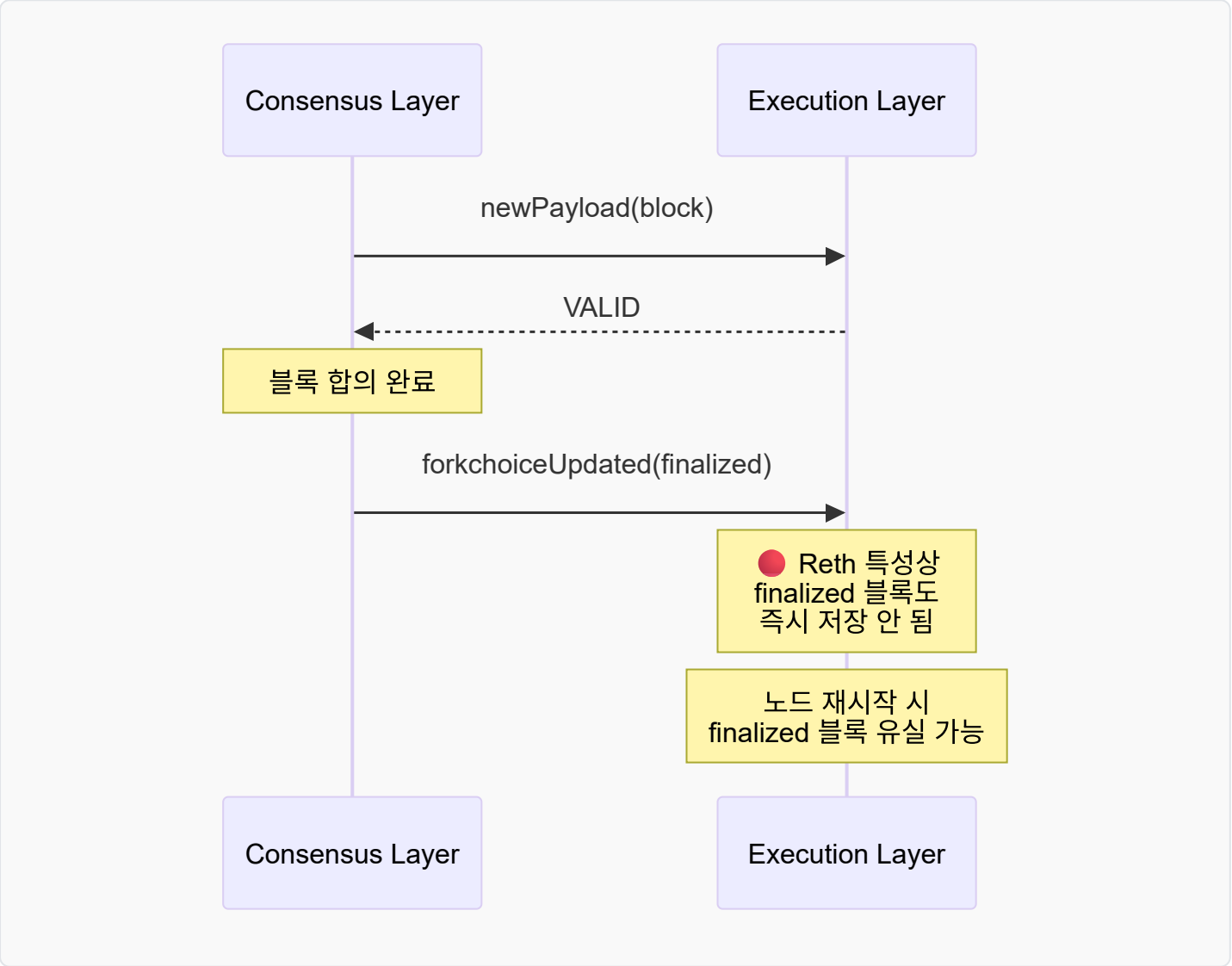
```
    // 1. Precompile로 SDK 상태 변경
    stakingPrecompile.delegate(validator, amount);  // 커밋됨!

    // 2. 이후 revert
    revert("Attack");

    // 결과: SDK 상태 변경은 유지, EVM 상태만 롤백
}
```

6.2 Separated Execution의 보안 고려

컨센서스 불일치 가능성



> "Reth does not immediately store on disk every block it received from its consensus layer... This is risky with Berachain, where you enjoy the single block finality offered by CometBFT."
> — BeaconKit Release Notes

BeaconKit은 이를 해결하기 위해 Reth에 특수 플래그를 설정하도록 권장합니다.

6.3 공통 보안 권장사항

영역	권장사항
가스 관리	EVM과 Cosmos 가스를 통합 관리하는 레이어 구현
상태 원자성	Precompile 호출 시 SDK 상태도 저널 시맨틱 적용
클라이언트 동기화	Separated 방식에서 EL finality 설정 검증
감사	두 시스템 인터페이스 집중 감사

---

7. 향후 전망

7.1 수렴 vs 분화

Cosmos EVM 생태계는 두 가지 방향으로 진화하고 있습니다:

수렴 경향:

- Engine API 기반 접근법의 표준화
- ABCI 2.0 활용 패턴의 공유
- 보안 모범 사례의 축적

분화 경향:

- 각 체인의 고유한 요구사항 (PoL, 롤업 등)
- IBC 통합 깊이의 차이
- 성능 vs 호환성 트레이드오프 선택

7.2 주목할 발전

1. **Cosmos EVM의 개선**: Evmos 팀의 지속적인 Precompile 원자성 해결 노력
2. **BeaconKit의 IBC**: BeaconKit에서 IBC 지원 가능성
3. **롤업 생태계**: OPinit, Rollkit 기반 L2 확산
4. **새로운 실행 클라이언트**: Monad, reth 등 최적화된 클라이언트 통합

---

8. 결론

Cosmos SDK 위에서 EVM을 구동하는 것은 "두 세계의 장점을 결합"한다는 매력적인 비전을 가지고 있지만, 실제 구현은 수많은 기술적 도전을 수반합니다.

Embedded EVM 방식 (Cosmos EVM, Polaris, MiniEVM)은 Cosmos SDK의 모듈성과 IBC를 완전히 활용할 수 있지만, StateDB 재구현, 가스 모델 불일치, Precompile 원자성 등의 복잡한 문제를 해결해야 합니다.

Separated Execution 방식 (Octane, BeaconKit)은 Engine API를 통해 검증된 EVM 클라이언트를 그대로 사용함으로써 호환성과 유지보수 문제를 해결하지만, Cosmos 생태계와의 깊은 통합은 포기해야 합니다.

> "Despite the Cosmos-SDK's goal of allowing for interoperable modules, we found that the coupling of these modules made implementing things the way we wanted to be extremely difficult. The way the SDK is currently implemented is very opinionated and leaves little to no flexibility."
> — Berachain Blog

어떤 접근법이 "더 낫다"고 단정하기는 어렵습니다. 각 프로젝트의 목표와 제약 조건에 따라 최적의 선택이 달라집니다. 중요한 것은 각 접근법의 트레이드오프를 정확히 이해하고, 해당 프로젝트의 요구사항에 맞는 아키텍처를 선택하는 것입니다.

참고 자료

공식 문서

- Cosmos EVM: https://evm.cosmos.network
- Ethermint: https://docs.ethermint.zone
- BeaconKit: https://docs.berachain.com
- Omni/Octane: https://docs.omni.network
- Initia/MiniEVM: https://docs.initia.xyz

GitHub 저장소

- cosmos/evm: https://github.com/cosmos/evm
- evmos/ethermint: https://github.com/evmos/ethermint
- berachain/polaris: https://github.com/berachain/polaris
- berachain/beacon-kit: https://github.com/berachain/beacon-kit
- omni-network/omni: https://github.com/omni-network/omni
- initia-labs/minievm: https://github.com/initia-labs/minievm

보안 연구

- CertiK: EVM-Cosmos Convergence Research
- Code4rena: Initia Cosmos Audit Report
- Zellic: Evmos Security Analysis